

Objective.....	2
Features Implemented	2
Challenges Encountered	3
Tools and Technologies Used	4
1. React.js:	4
2. Material-UI:.....	4
3. React Query:.....	4
4. Redux:.....	5
5. TypeScript:	5
6. React Modal:.....	5
7. Axios:	5
8. SCSS (Sass):	5
9. Git:.....	6
Code Structure and File Organization	6
Deployment and Hosting:	7
Conclusion	9

Frontend Implementation of Call Management Dashboard

Objective:

To develop a responsive and interactive frontend interface for managing call activities, enabling users to view, filter, group, select, archive, and add notes to calls efficiently.

Features Implemented

- **Call Table with Grouping and Filtering:**
 - Implemented a table to display call data, which can be grouped by parameters like **call type** or **date**.
 - Calls can be filtered based on **status** and **grouped** by the selected category (e.g., **call type**, **from**, **date**).
- **Row Selection and Bulk Actions:**
 - Added functionality for selecting individual calls by clicking checkboxes.
 - Implemented a “**Select All**” checkbox to select/deselect all calls in the current view.
 - Users can perform **bulk archive** actions on selected calls.
- **Archiving Calls:**
 - Implemented the ability to archive calls directly from the table by clicking an **Archive** button.
 - Calls are archived on the backend, and the frontend reflects changes dynamically.
- **Add Notes to Calls:**
 - Integrated a modal for adding/editing notes for individual calls.
 - Each call has a button that opens the modal, allowing users to input and save notes for that call.
- **Loading and Error Handling:**
 - Added loading indicators while calls are being fetched or archived.
 - Implemented error handling to ensure the UI gracefully handles API errors and retries if necessary.
- **Pagination Controls:**

- Pagination controls have been implemented to handle large sets of calls.
- Users can navigate through pages of calls using pagination buttons.
- **UI Enhancements with Material-UI:**
- Used Material-UI components like **Table**, **Checkbox**, **Button**, and **CircularProgress** for a consistent UI/UX.
- Styled the components for a clean and user-friendly interface.

Challenges Encountered

1. Handling Dynamic Row Selection and Bulk Actions:

- a. **Challenge:** Managing row selection dynamically, especially with bulk actions (select all, unselect all) and ensuring the table reflects the correct selection state after an action like archiving.
- b. **Solution:** I utilized React's `useState` to manage the selected rows and implemented logic to toggle the selection state based on user actions. The "Select All" checkbox logic was implemented by checking if all rows are selected, allowing the toggle functionality.

2. Archiving Calls and Optimistic UI Updates:

- a. **Challenge:** Implementing optimistic updates for archiving calls to provide a smooth user experience while ensuring backend consistency.
- b. **Solution:** I used `useMutation` from React Query to perform the archiving operation. The optimistic update ensured that the UI immediately reflected the archived state of calls, even before the backend confirmed the action. This involved handling `onMutate`, `onError`, and `onSettled` for error handling and data consistency.

3. Handling Pagination with Grouped Data:

- a. **Challenge:** Ensuring that pagination works correctly when calls are grouped by certain categories (e.g., date, call type). Grouped data can affect how pages are calculated and displayed.
- b. **Solution:** I implemented pagination by calculating the total number of pages based on the total count of calls. I made sure that the grouped data is paginated correctly by keeping track of the current page and loading data in chunks to avoid performance issues.

4. Modal for Adding Notes:

- a. **Challenge:** Managing the modal for adding/editing notes and ensuring it opened with the correct information when a call was selected.
- b. **Solution:** I utilized React's `useState` to manage the modal's open/close state and passed the selected call's data to the modal component. This ensured the modal was pre-populated with any existing notes and updated correctly after saving.

Tools and Technologies Used

1. *React.js:*

- **Description:** A JavaScript library for building user interfaces. React is used to build the component-based structure of the application, making it easy to manage UI states and handle reactivity in the app.
- **Reason for Use:** React is ideal for building interactive UIs with complex states, like the one required for the table that lists calls. The component-based architecture allowed for better maintainability and reuse of code.

2. *Material-UI:*

- **Description:** A popular React UI framework that provides a set of pre-designed and customizable components.
- **Reason for Use:** Material-UI components like `Table`, `Checkbox`, `Button`, and `CircularProgress` were used to speed up UI development and maintain a consistent design. It also allowed for better accessibility and responsiveness across devices.

3. *React Query:*

- **Description:** A data-fetching library for React that simplifies the process of fetching, caching, and synchronizing server state.
- **Reason for Use:** React Query was used to handle the fetching and caching of the calls data and archiving operations. It provided hooks such as `useMutation` and `useQuery`, allowing easy integration with the backend, automatic refetching, and error handling.

4. Redux:

- **Description:** A state management library for JavaScript applications, commonly used with React for managing global application state.
- **Reason for Use:** Redux was used to store and manage the authentication token (authToken). This allowed for consistent handling of authentication across different components without prop-drilling.

5. TypeScript:

- **Description:** A statically typed superset of JavaScript that helps catch errors early through type-checking.
- **Reason for Use:** TypeScript was used to improve code quality and maintainability by providing type safety, which helped prevent runtime errors and made the development process more predictable.

6. React Modal:

- **Description:** A React library for creating modal dialogs.
- **Reason for Use:** The modal was used to handle the "Add Note" functionality, allowing users to easily open a form to add or edit notes for a particular call.

7. Axios:

- **Description:** A promise-based HTTP client for the browser and Node.js.
- **Reason for Use:** Axios was used to interact with the backend API, including making requests to fetch calls data and archive calls. It provided a simple API for making HTTP requests and handling responses.

8. SCSS (Sass):

- **Description:** A CSS preprocessor that extends CSS with features like variables, nested rules, and mixins.
- **Reason for Use:** SCSS was used for styling the application. It allowed for more organized, modular, and maintainable styles, especially for large projects like this one.

9. Git:

- **Description:** A distributed version control system.
- **Reason for Use:** Git was used for version control to track changes, collaborate, and revert to previous versions when necessary.

Code Structure and File Organization

```
src/  
|  
├─ components/  
|   ├─ CallRow/  
|   |   ├─ CallRow.tsx  
|   |   ├─ CallRow.scss  
|   |   └─ index.ts  
|   └─ CallGroup/  
|       ├─ CallGroup.tsx  
|       ├─ CallGroup.scss  
|       └─ index.ts  
|   └─ ...  
|  
├─ hooks/  
|   ├─ useArchiveCall.ts  
|   ├─ useGroupCalls.ts  
|   └─ ...
```

```
|
|
| └─ interfaces/ <-- New folder for interfaces
|
|   └─ Call.ts <-- Interface for Call object
|
|   └─ CallGroup.ts <-- Interface for CallGroup props
|
|     └─ ...
|
|
| └─ utils/
|
|   └─ groupCalls.ts
|
|     └─ formatDate.ts
|
|
| └─ api/
|
|   └─ callApi.ts
|
|     └─ index.ts
|
|
| └─ styles/
|
|   └─ global.scss
|
|     └─ mixins.scss
|
|
| └─ App.tsx
|
|   └─ index.tsx
```

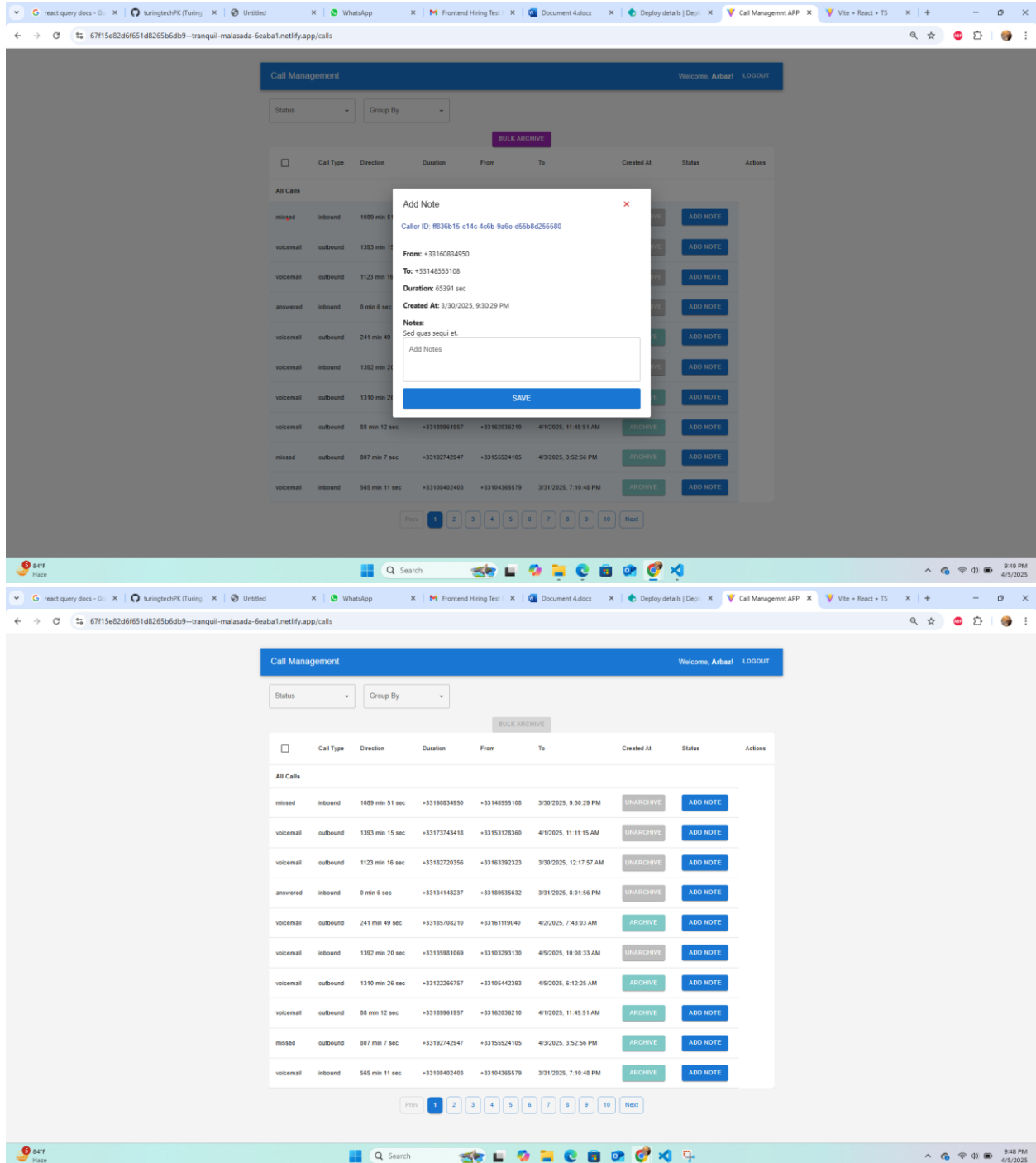
Deployment and Hosting:

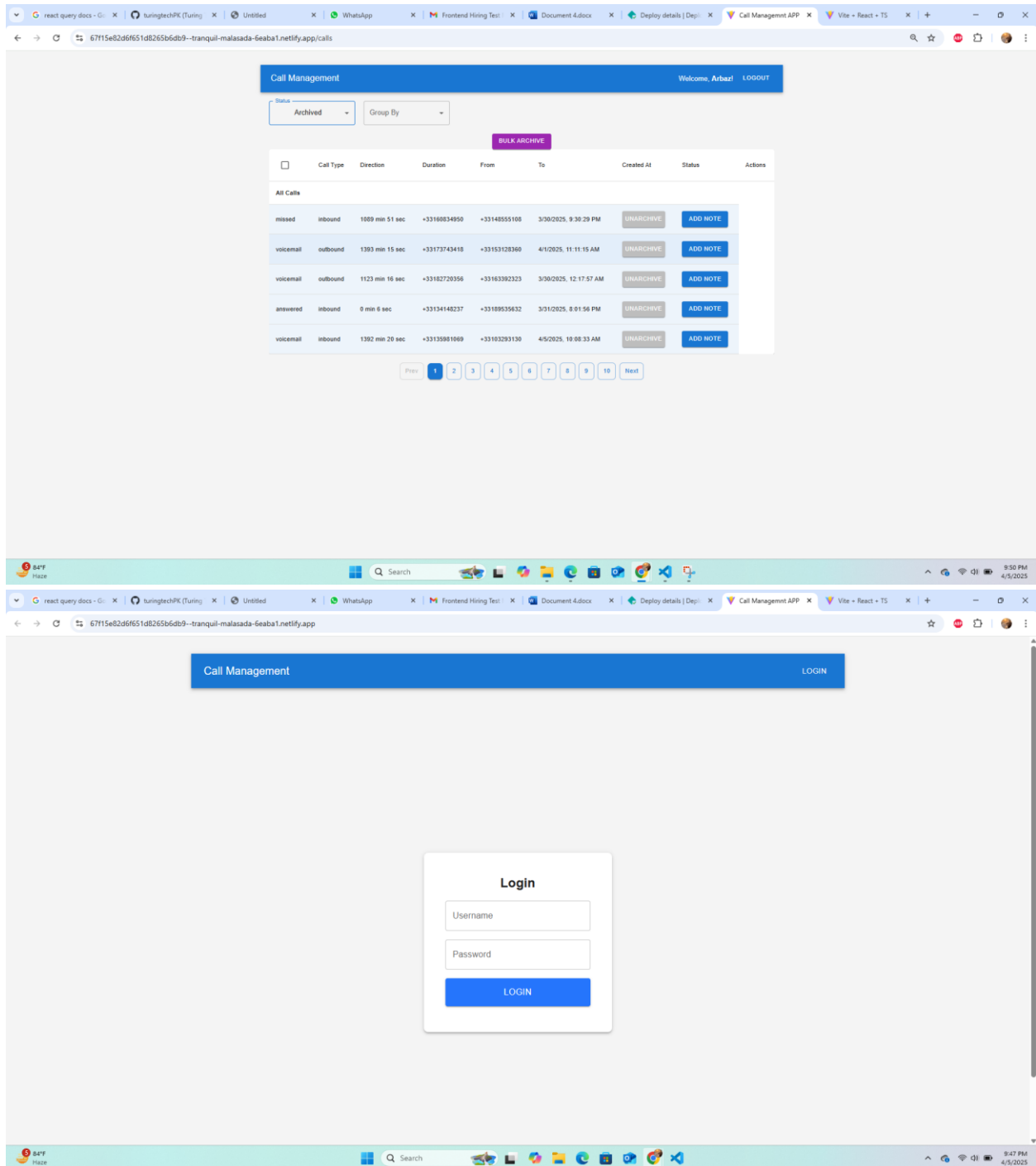
The frontend application was deployed to **Netlify** by manually uploading the build files. After running the `npm run build` command, the generated static files were uploaded to Netlify, which then served the application efficiently via its global CDN.

Here is link to App.

<https://67f15c8a24495029b39fe6bd--dancing-selkie-7fc083.netlify.app/>

Screenshots of the Application





Conclusion

In this task, I developed a feature-rich Call Management Table using React, Material UI, and Redux. The table allows users to filter, group, and manage calls, with smooth data fetching and a loading state displayed within the table body. I implemented pagination to

handle large datasets and ensured the table is fully responsive, providing a seamless experience across devices. With hooks for data fetching and actions, and Redux for state management, the table offers an efficient and user-friendly solution for managing call data.